# 임다영 Backend Developer Portfolio

**Email** · imlimdayoung@gmail.com **GitHub** · github.com/dlaekduddlaekdud **Phone** · 010-5933-4781

> **업무 규칙을 도메인 모델과 트랜잭션 경계로 구현하는 백엔드 개발자**
>
> 현장 업무를 **상태 전이와 변경 이력이 보존되는 도메인**으로 설계하고, 비관적 잠금·단일 트랜잭션·DB 제약을 각각의 방어선으로 두어 동시 요청과 중간 실패에도 정합성이 유지되는 시스템을 **실제 DB 기반 테스트**로 검증합니다.

## CORE SKILLS

| | |
|---|---|
| **Backend** | Java 21, Spring Boot, Spring Security, JPA, Validation, OpenAPI |
| **Database** | MySQL 8.4, PostgreSQL, Flyway — 트랜잭션·잠금·제약 기반 무결성 설계 |
| **Test** | JUnit 5, MockMvc, Testcontainers, Vitest, Playwright |
| **Infra** | Gradle, Docker Compose, GitHub Actions, Actuator |
| **Additional** | TypeScript, Next.js, React, Supabase, Python |

## PROJECT OVERVIEW

### 학원 근무·급여 정산 시스템

조교 최대 23명분의 실제 데이터 · 1개월 운영

**월 정산 시간 2시간 → 20분**

기획·개발·배포·운영 1인 담당 · **상세 보기**

### 구매·월 마감 관리 시스템

구매요청부터 입고·송장·사후 정정까지 구현

**실패 원인을 보존하는 월 마감 · MySQL 테스트 139개**

백엔드 도메인·API·DB·테스트·CI 담당 · **상세 보기**

### Study Gather · 스터디 모집 서비스

팀 통합 문제를 API 계약과 자동 검증으로 재설계

**동시 승인 20요청 × 20회 · 매 회 1건만 성공**

v1 5인 팀 → v2 1인 리빌드 · **상세 보기**

### 연구 데이터 검증 대시보드

Edge AI 칼만 필터 실험 데이터 검증

**자동화 테스트 53개 · 논문 오류 2건 발견·정정**

평가 파이프라인·대시보드 1인 담당 · **상세 보기**

PROJECT 01 · 대표 프로젝트

# 학원 근무·급여 정산 시스템

> **현장에서 직접 수행하던 반복 업무를 데이터 구조와 권한 규칙으로 옮긴 프로젝트**

| 2시간 → 20분 | 최대 23명 · 1개월 | 테스트 27개 통과 |
|---|---|---|
| 월 정산 소요 · 수작업 1:1 대조 → 시스템 입력 (1개월 운영 기준) | 조교 실제 근무·급여 데이터 운영 처리 | 단위·RPC 통합 23 + Playwright E2E 4 · Skip 0 |

| | |
|---|---|
| 기간 | 2026.03-2026.05 · 개발(03) → 실제 업무 적용(04) → 운영 종료(05) |
| 역할 | 기획·설계·구현·배포·운영 1인 담당 |
| Stack | PostgreSQL, RLS, Supabase Auth, Next.js 16, TypeScript strict, Vitest, Playwright |
| GitHub | github.com/dlaekduddlaekdud/academy-worklog-payroll |

## Background

약 4년간 영어학원 조교와 조교 총괄로 근무하며 스케줄·근무 기록·월별 급여 정산 검토를 담당했습니다. 개인 메시지와 구두로 전달된 기록을 대조하는 과정에서 네 가지 문제가 반복됐습니다.

- **원장 부재** — 기록이 흩어져 기억과 대화 내역에 의존
- **수동 집계** — 조교 최대 23명의 신고 시간 대조에 매월 약 2시간 소요
- **검증 부재** — 누락·중복·시급 적용 착오를 수작업으로 확인
- **확정 근거 미보존** — 급여 확정 후 당시 계산 근거 재현이 어려움

핵심 설계 질문은 **"어떤 기록이 언제 확정되고, 이후에도 당시 계산 근거를 재현할 수 있는가"**였습니다.

```
근무 기록 → 관리자 확인 → 수정·승인 → 월별 집계 → 급여 확정
```

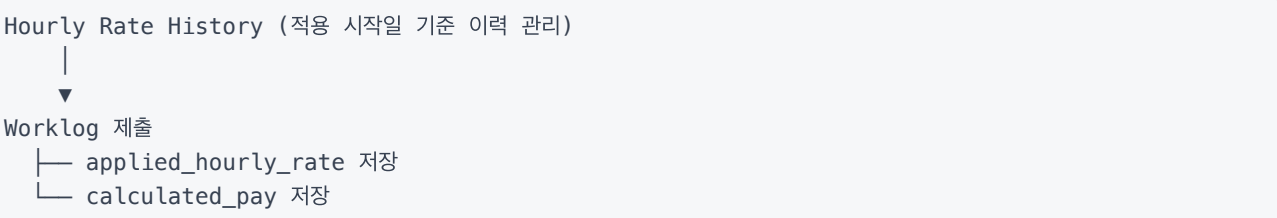
**실제 사용 범위 ·** 공식 인사 시스템 대체나 조교 전원의 직접 사용이 아닌 운영자용 정산 보조 도구입니다. 기존 방식으로 전달된 기록을 본인이 확인·입력해 약 1개월간 조교 23명분의 실제 데이터를 처리했습니다.

### 시급이 바뀌어도 과거 급여 계산 근거를 보존

**PROBLEM**

현재 시급만 참조해 급여를 계산하면 이후 시급이 변경됐을 때 과거 급여까지 달라질 수 있습니다.

**DECISION**




시급을 **적용 시작일 기준 이력 테이블**로 관리하고, 근무 기록 제출 시점의 적용 시급과 계산 금액을 해당 기록 행에 스냅샷으로 저장했습니다.

> **대안** 조회 시 이력 조인으로 당시 시급 역산
> **기각 이유** 이력 수정·삭제에 과거 급여가 종속됨. 정규화를 일부 포기하고 기록 행에 계산 근거를 고정.

**검증:** 급여 계산 및 월 경계

PROJECT 01 · 정산 확정과 데이터 보호

### 검사와 확정을 하나로 묶어 정산 불일치를 방지

**PROBLEM**

초기 구현은 대기 기록 확인 → 급여 집계 → 월 정산 확정 연산이 각각 독립적으로 실행돼, 검사와 확정 사이에 다른 기록이 승인되면 확정 금액과 실제 근무 기록이 달라질 가능성이 있었습니다.

**DECISION**

```
finalize_payroll()
  대기 기록 검증 → 급여 집계 → 확정 상태 변경 → COMMIT
```

세 연산을 PostgreSQL `finalize_payroll` 함수 하나로 옮기고 애플리케이션은 RPC 한 번만 호출하도록 변경했습니다. 요약 행을 `FOR UPDATE`로 잠그고, 함수 내부에서 관리자 권한을 다시 확인해 **검사와 확정이 하나의 원자적 연산으로 처리되도록 구성**했습니다.

> **대안** 앱 계층에서 세 호출을 트랜잭션으로 묶기
> **기각 이유** 왕복 3회로 잠금 유지 시간이 늘고, DB 직접 접근 시 권한 검증이 우회됨.

**검증:** 대기 기록 존재 시 확정 차단 · 재확정 시 기존 금액 유지

---

### 우회 요청에서도 확정 월의 데이터 변경을 차단

**PROBLEM**

애플리케이션 검증만으로 확정 데이터를 보호하면, 검증을 거치지 않는 경로(직접 쿼리, 클라이언트 우회)에서 확정된 월의 기록이 변경될 수 있습니다.

**DECISION**

```
Application  사용자에게 적절한 오류 안내     — 첫 번째 방어선
Database     우회된 요청에서도 무결성 보호   — 마지막 방어선
```

확정 월의 근무 기록 변경은 PostgreSQL 트리거로 차단하고, 승인·반려 데이터에 대한 사용자 직접 수정은 RLS로 제한했습니다. 두 계층의 역할을 **"안내"와 "보호"로 분리**해 어느 한쪽이 뚫려도 확정 데이터가 바뀌지 않도록 구성했습니다.

**검증:** 확정 월 데이터 보호 · 비관리자 접근 거부

**LEARNED** 기능보다 **"언제 데이터가 확정되는가"**를 먼저 정의해야 한다 — 확정 시점이 정해지면 원본 불변성·이력·트랜잭션 경계가 그 시점을 기준으로 따라온다. 이후 Spring 프로젝트의 설계 출발점이 됨

PROJECT 02 · 대표 프로젝트

# 구매·월 마감 관리 시스템

> **기업 구매 업무를 단순 CRUD가 아닌 데이터 생명주기 관점에서 구현한 Spring Boot 프로젝트**

| **테스트 139개 통과** | **마감 실패 원인 보존** |
| --- | --- |
| 실제 MySQL 8.4 기반 · 실패 0 | 집계 롤백과 실패 이력을 별도 트랜잭션으로 처리 |

| | |
| --- | --- |
| 기간 | 2026.08-2026.09 |
| 역할 | 백엔드 도메인 설계, API·DB 구현, 테스트, 문서화, CI·로컬 환경 구성 |
| Stack | Java 21, Spring Boot 4.1.1, Spring Security, JPA, MySQL 8.4, Flyway |
| Test / Infra | JUnit 5, MockMvc, Testcontainers, Docker Compose, GitHub Actions |
| GitHub | github.com/dlaekduddlaekdud/procurement-system |

## Business Flow

```
REQUESTER  구매요청 작성 → 제출
    ↓
BUYER      승인 / 거절 → 발주 생성·발송
    ↓
부분입고 · 분할송장 접수 → 3-Way Matching → MATCHED / HOLD → 미착 원장
    ↓
ADMIN      수동 / Scheduler 월 마감 → 마감 스냅샷 보존 → REVERSAL / CORRECTION
```

구매요청 한 건의 생성에서 끝내지 않고 **입고·송장·원장·마감·사후 정정까지 데이터의 생명주기 전체**를 구현했습니다. 각 단계 경계에서 "어떤 상태 전이가 허용되는가"와 "실패 시 무엇을 남기는가"를 먼저 정의하고, 트랜잭션·잠금·DB 제약·감사 이력을 그 정의에 따라 서로 다른 역할로 배치했습니다.

### 분할 도착해도 발주 라인 누적값으로 정확히 대사

PROBLEM

한 번의 발주가 여러 번 입고되고 송장 역시 나뉘어 들어올 수 있어, 부분 송장을 발주 전체 금액과 직접 비교하면 정상 거래까지 가격 불일치로 판단할 수 있습니다.

DECISION

입고 라인과 송장 라인이 발주 라인을 직접 참조하도록 구성하고, 발주 라인을 집계 기준으로 누적 입고량과 누적 송장량을 계산해 `MATCHED / HOLD_QUANTITY / HOLD_PRICE` 상태로 관리했습니다. 추가 입고가 발생하면 기존 HOLD 건도 다시 대사해 **최종 상태가 도착 순서에 의존하지 않도록** 했습니다.

> **대안** 송장 접수 시 발주 문서 단위로 즉시 대사
> **기각 이유** 분할 도착을 표현하지 못해 정상 거래도 HOLD로 남음. 라인 간 정합성 검증 부담을 감수.

**검증:** 부분입고·분할송장 · 가격·수량 HOLD · 추가 입고 후 재대사

PROJECT 02 · 월 마감과 사후 정정

### 마감 실패와 재시도에도 이전 실패 원인을 보존

**PROBLEM**

현재 마감 상태와 마감을 실행했던 기록을 하나의 행에서 관리하면 재시도 과정에서 이전 실패 원인이 덮어써질 수 있습니다.

**DECISION**

```
close_period (현재 상태) : OPEN / CLOSED
close_run    (실행 이력) : RUNNING / SUCCESS / FAILED / SKIPPED
```

현재 상태와 실행 이력을 별도 엔티티로 분리했습니다. 마감 실행 시 기간 행을 비관적 쓰기 잠금으로 보호해 동시 마감을 직렬화하고, 실패하면 집계 결과는 롤백하되 `REQUIRES_NEW`로 실패 이력과 원인은 별도 트랜잭션에 보존했습니다. 이미 닫힌 기간의 요청은 결과를 변경하지 않고 `SKIPPED` 실행 이력만 남겨 재시도가 안전하도록 했습니다.

> **대안** 마감 상태와 마지막 실행 결과를 단일 행에 저장
> **기각 이유** 실패 원인이 덮어써지고 본 트랜잭션 롤백 시 이력도 사라짐. 기간 행만 잠가 상태와 이력을 분리.

**검증:** 동시 월 마감 · 마감 실패 후 재시도

---

### 원본을 수정하지 않고 마감 후 정정 이력을 보존

**PROBLEM**

마감 이후 원본 문서나 기존 원장 금액을 직접 수정하면 과거 마감 결과를 재현하기 어렵습니다.

**DECISION**

```
OPEN PERIOD    원본 → CANCELLED → CANCEL_OFFSET
CLOSED PERIOD  Original Ledger → REVERSAL → CORRECTION
```

열린 기간의 취소와 닫힌 기간의 회계 정정을 분리했습니다. 마감 후에는 기존 원장을 직접 수정하지 않고 **반대 금액의 REVERSAL과 수정된 CORRECTION 행을 추가**하며, 실행자와 사유는 append-only 감사 로그에 기록해 변경 경로를 남겼습니다.

> **대안** 원본 UPDATE 후 변경 전 값을 이력 테이블에 저장
> **기각 이유** 재현에 이력 역추적이 필요하고 누락 시 복구 불가. 원장 자체를 append-only로 유지.

**검증:** 중복 정정

---

## Concurrency & Integrity

| Application | Database | Schema |
|---|---|---|
| REQUESTER · BUYER · ADMIN 역할 분리<br>Controller @PreAuthorize<br>Service 계층 역할·소유권 검증 | 문서번호 / 마감 기간 / 실행 attempt / 정정 연결 UNIQUE<br>수량·금액·상태 CHECK<br>비관적 잠금 | Flyway V1 ~ V15<br>Hibernate ddl-auto=validate |

인가는 애플리케이션이, 최종 무결성은 DB가 담당하도록 계층별 방어선을 분리했습니다. 애플리케이션 검증은 우회될 수 있다는 전제로 **UNIQUE·CHECK 제약을 최종 경계**로 두고, Flyway와 `ddl-auto=validate`로 코드와 스키마의 불일치를 기동 시점에 차단했습니다.

**Validation** · 실제 MySQL 8.4 Testcontainers · 테스트 파일 40개 / 139개 통과 · Flyway Migration 15개 · 설계 근거는 `docs/adr/`에 ADR로 기록

**LEARNED** 업무 단계 사이의 **상태 이동과 실패 시 보존할 기록**이 핵심 — 트랜잭션은 결과를, 이력은 실패를 지키도록 역할을 분리해야 재시도 가능한 시스템이 된다

PROJECT 03 · 대표 프로젝트

# Study Gather · 스터디 모집 서비스

> **팀 통합에서 겪은 문제를 개인의 실수가 아닌 검증 구조의 부재로 진단하고, 규칙 위반이 자동으로 발견되는 구조로 재설계한 프로젝트**

| 동시 승인 20요청 × 20회 | 전체 롤백 | 계약 CI |
|---|---|---|
| 마지막 한 자리 동시 승인 20요청 × 20회, 매 회 승인 1건만 성립 | 중간 실패 주입 시 여러 테이블 상태 함께 복원 | OpenAPI 명세와 프론트 타입 불일치 시 CI 실패 |

| | |
|---|---|
| **v1** | 2025.10-2025.11 · 5인 팀 프로젝트 |
| **v2** | 2026.08 · 백엔드·프론트·DB·CI·배포 1인 리빌드 |
| **Stack** | Java 21, Spring Boot 4.1, Spring Security, JPA, MySQL 8.4, Flyway / Front: React, TypeScript, Vitest |
| **GitHub** | github.com/dlaekduddlaekdud/study-gather-v2 |
| **Live Demo** | study-gather-v2.ekdud0112.workers.dev · 회원가입 없이 목록·상세 조회, 데모 버튼으로 운영자·참여자 기능 체험 |

## From v1 to v2

v1에서는 문서로 공유한 API 규약과 실제 구현이 통합 단계에서 어긋났습니다. **필드명이 서로 달랐고, 검증 규칙이 한쪽에만 있었으며, 스키마가 코드 변경에 따라 조용히 바뀌었습니다.** 각각을 고치는 대신 공통 원인을 찾았습니다 — 규약 위반을 사람이 발견하는 구조였다는 것. v2는 모든 규약에 위반 시 자동으로 실패하는 검증 지점을 하나씩 붙이는 방향으로 재설계했습니다.

| v1 문제 | v2 변경 | 자동 검증 |
|---|---|---|
| API 필드·주소 수동 연결 | OpenAPI 기반 타입 생성 | 계약 CI |
| Controller 검증 누락 | DTO Validation | MockMvc |
| ddl-auto=update | Flyway + validate | CI MySQL |
| 마지막 한 자리 동시 승인 | 비관적 잠금 | 동시성 테스트 |
| 승인 도중 일부 데이터 반영 | 단일 트랜잭션 | 실패 주입 테스트 |

## API Contract

```
Spring OpenAPI → TypeScript Type 생성 → Frontend 사용 → GitHub Actions → 차이 발생 시 CI 실패
```

문서로만 API 계약을 공유하는 대신, 명세에서 프론트 타입을 생성하고 변경 여부를 CI에서 검사하도록 구성했습니다.

PROJECT 03 · 동시 승인과 롤백 검증

### 마지막 한 자리의 동시 승인에서 정원 초과를 방지

**PROBLEM**

정원이 한 자리 남은 상태에서 여러 요청이 동시에 승인되면 각 트랜잭션이 같은 승인 인원을 읽고 모두 통과할 수 있습니다.

**DECISION**

```
신청 행   PESSIMISTIC_WRITE → PENDING 확인
스터디 행 PESSIMISTIC_WRITE → 남은 정원 확인
신청 APPROVED → 멤버 생성 + 인원 증가 → COMMIT
```

신청 행을 먼저 잠가 **같은 신청의 중복 상태 전이를 차단**하고, 스터디 행을 잠가 **동일 스터디의 정원 검증을 직렬화**했습니다. 잠금 순서를 신청 → 스터디로 고정해 두 자원을 잠그는 트랜잭션 간 교착 위험을 줄였습니다.

> **대안** 버전 컬럼을 사용하는 낙관적 잠금
> **기각 이유** 마지막 자리처럼 충돌이 집중되는 구간에서 재시도 부담이 커짐. 비관적 잠금으로 승인 직렬화.

---

**CONCURRENCY TEST**

잔여 정원 1명 상태에서 동시 승인 요청 20개 × 반복 실행 20회를 검증했습니다. 확인 기준은 두 층입니다 — 각 반복마다 **승인 성공 1건 / 정원 초과 실패 19건**이 나오는가, 그리고 트랜잭션 종료 후 **승인된 신청 수·스터디 승인 인원·실제 멤버 수** 세 값이 모두 일치하는가. 응답 결과만 세면 통과했더라도 테이블 간 불일치가 남을 수 있어, 상태 검증을 응답이 아니라 DB 최종 상태에 뒀습니다.

**TRANSACTION ROLLBACK**

승인 과정 중 멤버 저장에서 강제로 예외를 발생시킨 뒤, **신청 상태 PENDING · 승인 인원 개설자 1명 · 신청자 Member 없음** 세 가지 최종 상태를 다시 조회해 복원 여부를 확인했습니다. 예외가 던져졌는지가 아니라 **여러 테이블의 상태가 함께 이전으로 돌아갔는지**를 검증 대상으로 삼았습니다.

**LEARNED** 규칙을 문서에 적는 것보다 **규칙이 깨졌을 때 자동으로 발견되는 지점**을 만드는 것이 중요 — 테스트·CI를 설계 규칙을 강제하는 장치로 사용

PROJECT 04 · 보조 프로젝트

# 연구 데이터 검증 대시보드

> **논문 정의와 실제 실험 데이터를 자동으로 검증하는 연구 데이터 파이프라인**

| **테스트 53개** | **2건 발견·정정** |
| --- | --- |
| 자동화 테스트 (지표 계산·CSV 스키마·Ground Truth 복원 ·Ablation 판정) | NIS 95% 신뢰구간 하한 오기 · Ablation 실측값과 본문 해석 불일치 |

| | |
| --- | --- |
| **기간** | 2026.03-2026.08 |
| **역할** | Python 평가·검증 파이프라인 및 웹 대시보드 설계·개발·배포 1인 담당 |
| **Stack** | Python, Next.js 15, TypeScript, PapaParse, Zustand, Recharts, Vitest |
| **Background** | Edge AI 기반 적응형 칼만 필터 2인 공동 졸업연구 (2026.03-2026.06) |
| **GitHub** | **github.com/dlaekduddlaekdud/edge-ai-kalman-dashboard** · Live: **edge-ai-kalman-dashboard.vercel.app** |

## 담당 범위

| 공동연구자 담당 | 본인 담당 |
| --- | --- |
| TinyML 모델 설계·학습<br>INT8 양자화<br>STM32 펌웨어 추론 통합 | 논문 1·2·4장 · 실험 시나리오 · 평가 지표 체계<br>Python 평가·검증 파이프라인<br>웹 대시보드 설계·개발·배포 |

## Data Validation

실험 CSV는 기본 25컬럼 형식과 TinyML 결과가 추가된 28컬럼 형식으로 구성됩니다. 전체 컬럼 개수만 기준으로 파일 종류를 판단하면 잘못된 입력을 정상 파일로 인식할 가능성이 있어 **필수 헤더의 존재 여부를 기준으로 스키마를 판별**했습니다.

```
CSV Input → Required Header Validation → Schema Detection → Metric Calculation → Scenario Comparison → Visualization
```

## Validation Result

자동화 테스트 **53개**를 구성했습니다. 검증 범위: 지표 계산 · CSV 스키마 · 차단 탐지 · Ground Truth 복원 · Ablation 파싱 · 성능 열화 판정.

논문 정의와 실제 코드·산출 데이터를 교차 확인하는 과정에서 **NIS 95% 양측 신뢰구간 하한 오기와 Ablation 실측값과 본문 해석 불일치**를 발견하고 실측 데이터 기준으로 정정했습니다. 두 건 모두 시각화 결과가 아니라 논문 정의·코드·실험 산출물의 교차 검증 과정에서 드러났습니다.

**LEARNED** 시각화보다 **입력 데이터·수식·코드·논문 해석이 동일한 정의를 쓰는지** 먼저 검증해야 한다 — 검증 없는 시각화는 틀린 데이터를 설득력 있게 보여줄 뿐

## ADDITIONAL PROJECT

**TASKCARD** — Flask 기반 메모 카드 관리 서비스 · AdvICE 학회 학과 장려상 (2024.11.29)

2024.11 · 4인 팀 프로젝트(백엔드 2 · 프론트엔드 2) · 백엔드 담당

Flask · SQLAlchemy

- Blueprint 기반 Card CRUD API 구현 · SQLAlchemy 모델 구현
- 입력 검증 및 예외 처리 · 예외 발생 시 DB rollback 적용

GitHub: **github.com/taskcard/taskcard**

## EXPERIENCE

**베토영어학원** | 조교 → 조교 총괄 · 2021.12-2026.05 · 약 4년

- 약 250명 규모 학원에서 초·중·고등관 업무, 마지막 기준 조교 23명의 스케줄·근무 기록·급여 정산 검토 담당 — 이 업무를 **학원 근무·급여 정산 시스템**으로 직접 시스템화
- 신규 조교 30명 이상 인수인계 및 Notion 업무 매뉴얼 작성, 학생관리 스프레드시트의 출석·과제·재시험·누적경고 항목을 담당자 교체 후에도 이어지는 기록 구조로 정비

현장에서 업무 규칙과 예외가 반복되는 것을 4년간 직접 겪은 경험이, 이후 시스템 설계에서 **데이터의 확정 시점과 변경 이력을 기능보다 먼저 정의하는 방식**으로 이어졌습니다.

## EDUCATION & CERTIFICATION

| | |
|---|---|
| **Education** | 한국외국어대학교 정보통신공학과 · 2022.03-2027.02 졸업예정 |
| **Certification** | SQLD · 정보처리기사 (2026.09) · TOEIC Speaking IH (140) |
| **Award / Activity** | AdvICE 정보통신공학과 개발 학회 · TASKCARD 프로젝트 학과 장려상 |

## AI 활용 방식

| 직접 수행 | 위임 |
|---|---|
| 도메인 규칙·상태 전이 정의 | 보일러플레이트·설정 |
| 트랜잭션 경계와 잠금 범위 결정 | CRUD·DTO 초안 |
| 테스트 시나리오 설계 (동시성·실패·권한) | 마이그레이션 초안·프론트 폼 |
| ADR 작성과 설계 근거 기록 | 반복 리팩터링 |

**설계 판단과 검증 기준은 위임하지 않았습니다.** 생성된 코드는 실제 DB 기반 테스트를 통과해야 반영했고, 통과하지 못한 코드는 원인을 직접 추적해 테스트 기준을 먼저 보강했습니다.

현장 업무를 운영 도구로 바꾼 경험을 바탕으로, 복잡한 업무 규칙과 예외를 명확한 도메인으로 구현하는 백엔드 개발자로 성장하고자 합니다.